

1.实验目的

- 1) 学习使用简化段定义格式进行汇编语言程序设计
- 2) 练习相关伪指令
- 3) 熟练进行程序流程操作

2.实验环境

- 操作系统：Windows（Windows 11）
- 运行环境：DOSBox 模拟器（模拟 x86 实模式）
- 实验与调试工具：
 - DEBUG（DEBUG.exe，用于调试 16 位汇编 COM 程序）
 - 必要时可使用 MASM、LINK 编译汇编程序

3.实验内容

- 1、从键盘输入一个字符串：对其中的所有字母进行排序，将结果输出；
如，输入：software 输出：erawtfos

4.实验具体实现

代码

```
1  DATA SEGMENT
2      ; 提示信息
3      MSG_INPUT  DB 'Please input a string: $'
4      MSG_OUTPUT DB 0DH, 0AH, 'Sorted result: $'
5
6      ; 定义输入缓冲区（DOS 0Ah 功能调用格式）
7      ; 第1字节：缓冲区最大长度
8      ; 第2字节：实际输入长度（系统自动回填）
9      ; 后续：实际存储字符串
10     BUF_MAX     DB 100
11     ACT_LEN     DB ?
12     STRING      DB 100 DUP(?)
13
14     ; 定义桶（Buckets）
15     BUCKETS     DB 256 DUP(0)
16 DATA ENDS
```

```

17
18 CODE SEGMENT
19     ASSUME CS:CODE, DS:DATA
20 START:
21     ; 初始化数据段
22     MOV AX, DATA
23     MOV DS, AX
24
25     ; 1. 输出提示信息
26     LEA DX, MSG_INPUT
27     MOV AH, 09H
28     INT 21H
29
30     ; 2. 获取用户输入字符串
31     LEA DX, BUF_MAX
32     MOV AH, 0AH
33     INT 21H
34
35     ; --- 桶排序（计数）阶段 ---
36
37     ; 获取实际输入的长度
38     XOR CX, CX          ; 清空 CX
39     MOV CL, ACT_LEN     ; 将实际长度放入 CL
40     JCXZ PROGRAM_END   ; 如果长度为0（直接回车），则结束程序
41
42     LEA SI, STRING      ; SI 指向字符串首地址
43     LEA BX, BUCKETS     ; BX 指向桶的首地址
44
45 COUNT_LOOP:
46     MOV AL, [SI]        ; 取出一个字符
47     XOR AH, AH          ; 清空高位，使 AX = 字符的 ASCII 码
48     MOV DI, AX          ; 将 ASCII 码作为桶的索引（Offset）
49
50     ; 对应桶的计数 +1
51     INC BYTE PTR [BX+DI]
52
53     INC SI              ; 指向下一个字符
54     LOOP COUNT_LOOP     ; 循环直到处理完所有字符
55
56     ; --- 输出结果阶段 ---
57
58     ; 输出结果提示前缀
59     LEA DX, MSG_OUTPUT
60     MOV AH, 09H

```

```

61     INT 21H
62
63     ; 遍历 256 个桶
64     XOR CX, CX           ; CX 用作 ASCII 码计数器 (0 到 255)
65     LEA BX, BUCKETS     ; BX 重新指向桶首地址
66
67     PRINT_OUTER_LOOP:
68         MOV DI, CX       ; 当前 ASCII 码作为索引
69         MOV AL, [BX+DI]  ; 获取该字符出现的次数 (Count)
70
71         CMP AL, 0        ; 如果计数为 0
72         JE NEXT_CHAR    ; 跳过, 处理下一个 ASCII 码
73
74         ; 如果计数 > 0, 则打印该字符 AL 次
75         PUSH CX          ; 保存外层循环计数器 (当前的 ASCII 码)
76
77         MOV CL, AL       ; 将次数放入 CL 作为循环计数
78         MOV DX, DI       ; 将当前的 ASCII 码放入 DL (准备打印)
79         XOR CH, CH       ; 确保 CH 为 0
80
81     PRINT_INNER_LOOP:
82         MOV AH, 02H      ; DOS 21h 02号功能: 打印字符
83         INT 21H
84         LOOP PRINT_INNER_LOOP
85
86         POP CX           ; 恢复外层循环计数器
87
88     NEXT_CHAR:
89         INC CX           ; 下一个 ASCII 码
90         CMP CX, 256      ; 检查是否遍历完所有 256 个 ASCII
91         JNE PRINT_OUTER_LOOP
92
93     PROGRAM_END:
94         ; 退出程序
95         MOV AH, 4CH
96         INT 21H
97
98     CODE ENDS
99     END START

```

编译运行结果

```
D:\>ml lab7.asm
Microsoft (R) Macro Assembler Version 6.11
Copyright (C) Microsoft Corp 1981-1993. All rights reserved.

Assembling: lab7.asm

Microsoft (R) Segmented Executable Linker Version 5.60.339 Dec 5 1994
Copyright (C) Microsoft Corp 1984-1993. All rights reserved.

LINK : warning L4017: /r : unrecognized option name; option ignored
Object Modules [.obj]: /r lab7.obj
Run File [lab7.exe]: "lab7.exe"
List File [nul.map]: NUL
Libraries [.lib]:
Definitions File [nul.def]:
LINK : warning L4021: no stack segment

D:\>lab7.exe
Please input a string: software
Sorted result: aeforstw
D:\>
```

5. 实验分析与总结

5.1 算法实现分析

本次实验通过汇编语言实现了一个字符串排序程序。不同于常见的冒泡排序或选择排序，本程序采用了**桶排序**的思想，具体分析如下：

- **数据结构设计**：程序在数据段定义了 `BUCKETS DB 256 DUP (0)` 1，即开辟了 256 个字节的内存空间，对应 ASCII 码表的 0-255 个字符。
- **排序逻辑**：
 1. **计数阶段**：程序遍历输入字符串，将字符的 ASCII 码值存入 `AX`，并将其传送给 `DI` 作为索引（Offset）。利用指令 `INC BYTE PTR [BX+DI]` 2，直接在对应的“桶”中增加计数。这种方法只需遍历一次字符串，时间复杂度为 $O(n)$ 。
 2. **输出阶段**：程序使用了双重循环。外层循环 `PRINT_OUTER_LOOP` 遍历 0 到 255 所有可能的 ASCII 码；内层循环 `PRINT_INNER_LOOP` 根据桶中的计数值，重复打印该字符 3。
- **优势**：该算法避免了复杂的两两比较和交换操作，对于 ASCII 字符排序非常高效。

5.2 程序执行流程与指令应用

- **输入处理**：使用了DOS中断 `INT 21H` 的 `0Ah` 功能号读取带缓冲区的字符串 4，并利用 `ACT_LEN` 准确获取了用户实际输入的字符长度，避免了处理无效数据。
- **寻址方式**：程序中灵活运用了**基址变址寻址方式**（如 `MOV AL, [BX+DI]`）来访问桶数组，其中 `BX` 指向桶首地址，`DI` 动态指向当前 ASCII 码位置。
- **循环控制**：使用了 `LOOP` 指令控制计数循环，以及 `CMP` 和 `JNE` 指令控制输出循环，逻辑严密。

5.3 编译与调试结果分析

从编译运行截图来看，程序成功实现了预期功能：输入 "software" 后，成功输出了排序后的 "aeforstw"。

但在链接（LINK）过程中出现了一个警告：

- **警告信息**：`LINK: warning L4021: no stack segment`。
- **原因分析**：源代码中只定义了 `DATA SEGMENT`（数据段）和 `CODE SEGMENT`（代码段），没有显式定义堆栈段（`STACK SEGMENT`）。

5.4 实验心得

1. **加深了对内存操作的理解**：通过直接操作内存地址（桶数组）来实现排序，比高级语言更直观地理解了数据在内存中的存储方式。
2. **掌握了字符串处理技巧**：熟练了 `INT 21H` 的 `09H`（显示字符串）和 `0Ah`（缓冲输入）功能的调用规范。
3. **调试能力提升**：在实验过程中，认识到了定义完整段结构（包括堆栈段）的重要性，学会了通过编译器警告信息（Warning）来优化代码结构。